



BINV1053

4.1 - introduction aux services

Jérôme Plumat

Haute École Léonard de Vinci

2026





- 1 Introduction
- 2 Rappels
- 3 Introduction aux services
- 4 premier service : LoggerService
- 5 Services et gestion des erreur
 - Logger et Happy Path
- 6 Conclusion



1. Introduction

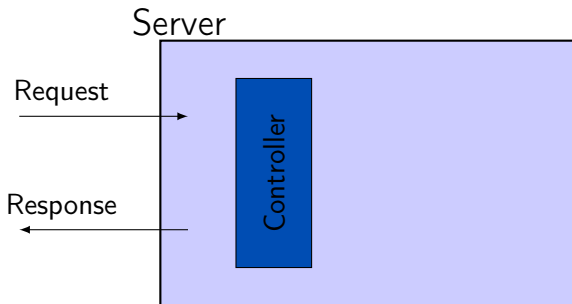


Objectifs

- Comprendre le principe de services et l'utiliser pour le `LoggerService` ;
- Comprendre et implémenter la notion de filtrage.

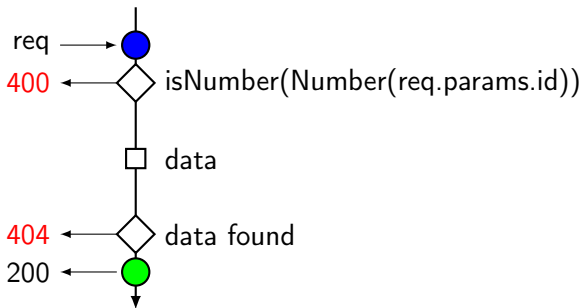


2. Rappels





Exemple : GET /doctors/1



La récupération des données (data) : **le service.**



3. Introduction aux services

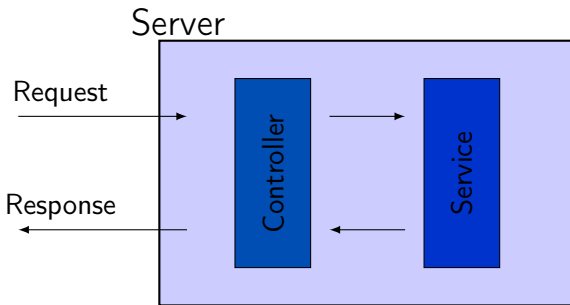


Jusqu'à présent, notre serveur est composé d'un seul élément : le contrôleur. Celui-ci est responsable de la gestion des routes, de la réception des requêtes et de l'envoi des réponses. Cependant, il est possible de déléguer certaines tâches à des **services**.

Cela permet d'obtenir un code plus modulaire, plus lisible et plus facile à tester. Bref, de meilleure qualité !



Introduction de la couche service (ou, couche *métier*) dans notre serveur.





Définition

Un **service** est une **classe** qui contient des méthodes (généralement statiques) qui effectuent des tâches spécifiques. Ces méthodes peuvent être appelées depuis n'importe quel composant de l'application.

Il existe différents services permettant différentes fonctionnalités, parmi lesquelles on retrouve classiquement :

- logger des évènements (**LoggerService**),
- gérer des ressources (**DoctorsService**, **PatientsService**, etc.),
- lire et écrire des données dans un fichier (**FileService**).



Pourquoi utiliser des services ?

- Pour séparer les tâches et les responsabilités (code unitaire) ;
- Pour réutiliser du code ;
- Pour rendre le code plus lisible ;
- Pour simplifier le travail d'équipe ;
- Pour faciliter les tests unitaires.



Dans le cadre de notre cours, les services seront implémentés dans des classes présentes dans le dossier `services`. Chaque service sera dans un fichier séparé. Ainsi le service qui sera responsable de la ressource `Doctor` sera dans le fichier `doctors.service.ts`. Le service responsable d'enregistrer les logs sera `logger.service.ts`.

Nous utiliserons des méthodes **statiques** pour les services. Cela permettra de les appeler sans devoir instancier la classe.

Les services

Exemple 1



```
1 export class LoggerService {  
2     static info(message: string): void {  
3         console.log(`[INFO]`, message);  
4     }  
5 }  
6
```

Dans cette exemple, la méthode `log` du service `LoggerService` permet d'afficher un message dans la console. Et l'on peut l'utiliser comme suit :

```
1 import { LoggerService } from './services/logger.service';  
2 // ...  
3 LoggerService.info('Hello World');
```



```
1 export class DoctorsService {  
2     static public getAll(): Doctor[] {  
3         const results: Doctor[] = [];  
4         // ...  
5         return results;  
6     }  
7 }
```

Ici un exemple très simple d'un service qui retourne la liste de tous les docteurs. On peut l'utiliser comme suit :

```
1 import { DoctorsService } from './services/doctors.service';  
2 // ...  
3 const doctors: Doctor[] = DoctorsService.getAll();
```



4. premier service : LoggerService



Le `LoggerService` est un service très simple qui permet d'enregistrer ce qui se passe au sein de notre serveur. Il est très utile pour débbugger notre application mais également dans son fonctionnement quotidien lorsqu'il est en production.

Notre `LoggerService` est un enregistreur d'évènements. Ces évènements sont appelés **logs**. L'enregistrement des logs peut se faire à différents endroits. Par exemple :

- dans la console,
- dans un fichier,
- dans une base de données.



Exemple de logs :

- "GET /doctors" : un client a contacté l'url `/doctors` avec la méthode GET ;
- "Doctor with ID 123 NOT FOUND" : une ressource n'a pas été trouvée ;
- "ERROR : file 'doctors.json' not found" : une erreur s'est produite lors de la lecture du fichier `doctors.json`.



Bien entendu, tous les logs précédemment vus n'ont pas la même importance. C'est pourquoi les logs sont classés en différents **niveaux**.

On distingue généralement les niveaux suivant :

- **debug** : pour les informations de debuggage, qui ne seront pas gardées lors de la mise en production ;
- **info** : pour les informations générales, d'importance mineure, généralement pas gardées en production ;
- **warning** : pour les avertissements, il ne s'agit pas d'une erreur mais d'un comportement inattendu mais pour lequel une solution a été trouvée, les logs sur ce canal sont toujours gardées en production ;
- **error** : pour les erreurs, elles sont toujours gardées en production, elles doivent déclencher une alarme sous une forme ou une autre ;



On voit que les différents niveaux de logs seront utilisés à différents moments du cycle de vie de notre serveur (production, développement, analyse d'erreurs, etc.).

On enregistre donc généralement les différents niveaux à différents endroits.



Dans le cadre de ce cours, nous allons nous contenter de deux niveaux de logs :

- **info** : que nous afficherons en console ;
- **debug** : que nous afficherons en console ;
- **error** : que nous afficherons en console et que nous enregistrerons dans un fichier.



5. Services et gestion des erreur



Les services introduisent un risque d'erreurs supplémentaires. En effet, ceux-ci étant généralement connecté à différents composants (système de fichiers, base de données, autre serveur, etc.), **ils peuvent générer des erreurs qui sont indépendantes de la requête HTTP émise par le client.**

Ces erreurs, générées par nos services, ne concernent pas ce client. Par exemple, on ne veut pas qu'un site internet affiche le fait qu'une certaine requête SQL ne s'est pas bien faite sur une table en donnant trop d'informations à l'utilisateur. Il est donc important de les gérer correctement.



Nous allons voir plus en détail, lors de la prochaine présentation comment faire cela. Mais, à ce stade, appelons cela des **erreurs internes** ou **internal error** une erreur qui n'est pas liée à la requête HTTP émise par le client et qui concerne notre système ou un élément extérieur.

Attention

Les erreurs provenant d'une mauvaise requête **et** les erreurs internes seront loggées sur le canal `error` de notre `LoggerService`.



Pour le moment :

Quand logger des erreurs internes ?

Tous les logs enregistré dans le canal `error` proviendront d'erreurs internes détectée par un bloc `try-catch` dans notre code.



Nous mettrons tout cela en pratique dans nos services au cours de cette semaine. Et nous verrons cela en détails dans la prochaine présentation.



6. Conclusion



- La couche service permet de séparer les tâches et les responsabilités ;
- Les services sont des classes qui contiennent des méthodes static qui effectuent des tâches spécifiques ;
- Le `LoggerService` est un service qui permet d'enregistrer des logs sur différents niveaux ;